

MockAI: A Multi-Agent LLM Framework for Automated Technical Interview Assessment with Real-Time Gaze-Based Proctoring

1st Renuka N

Artificial Intelligence And
Machine Learning
Kongu Engineering College
Perundurai, Erode
renuka66ksr@gmail.com

2nd Kannan N

Artificial Intelligence And
Data Science
Kongu Engineering College
Perundurai, Erode
kannanmese@gmail.com

3rd Karthikeyan T A

Artificial Intelligence And
Data Science
Kongu Engineering College
Perundurai, Erode
tak03793@gmail.com

4th Rithik R

Artificial Intelligence And
Machine Learning
Kongu Engineering College
Perundurai, Erode
rithikyalini@gmail.com

5th Sanjay K

Artificial Intelligence And
Machine Learning
Kongu Engineering College
Perundurai, Erode
ksanjayias@gmail.com

Abstract—Recruitment today requires a scalable, standardized, and objective process of candidate assessment. Interview-based processes face bottlenecks due to human subjectivity, resource limitations on the side of the interviewer, and impossibility of quality assurance for a large number of candidates in a limited timeframe. In this paper, we introduce MockAI – an AI-based mock interview solution which automates a fully automated three-stage evaluation process involving aptitude test, technical interview driven by AI and behavioral interview conducted by HR. We use LangChain-orchestrated LLM agents using Groq API with openai/gpt-oss-120b model with Contextualized Question Generation based on ConversationBufferMemory in order to provide coherent multistep conversation akin to an adaptive human-driven interview. Real-time proctoring is performed with a gaze-tracking OpenCV component with Haar Cascades with adaptive pupil centroid thresholding without the need of GPU or licensed software. An eight-agents modular architecture used in React 19, Flask 3.0, and MongoDB Atlas proves that multi-agents orchestration and classical computer vision can be implemented in an assessment platform on a consumer-grade hardware. Sub-second alert latency, LLM inference less than 500 ms per step and consistent numeric score out of 50 with regex cascading pipeline was proven empirically.

Keywords—multi-agent LLM; automated interview; gaze-based proctoring; LangChain; Groq API; OpenCV; Haar Cascade; recruitment automation; conversational AI; technical assessment; ConversationBufferMemory; regex score extraction; Flask; MongoDB Atlas; React

I. INTRODUCTION

The global recruiting market is facing increasing constraints from the limitations of scale in conducting interviews by humans. Enterprises doing mass recruiting are becoming increasingly constrained by the need to evaluate their candidates

in a consistent manner within very tight timelines. Technical evaluations have been limited by subjective evaluation, uncalibrated question difficulty, and an inherent inability to handle large numbers of candidates in a scalable manner. The recent rapid evolution of large language models has enabled the realization of dynamically generated conversational systems that are capable of generating new, topic-specific questions dependent on the candidate's responses [1].

Automated interviewing technology has so far largely addressed either question generation or proctoring of the candidates independently, rarely both. Rule-based question pools are highly consistent but not adaptive; they are unable to take advantage of what the candidate has done in a particular project or to explore a topic in which the candidate might be technically weak. Remote proctoring software like Proctorio depends on browser lockdown technologies, costly infrastructure partnerships, and invasive monitoring technologies that pose significant privacy concerns and are prohibitively expensive to deploy in an academic setting [2]. The lack of a coherent open-source framework combining LLM-enabled interview orchestration with lightweight proctoring poses a gap limiting the deployment of automated assessments.

MockAI bridges this gap via a highly modular eight agents pipeline orchestrating the end-to-end interview process from candidate registration to scorecard generation. Deployable on commodity hardware, the system does not require GPU support in clouds and is thus appropriate for use by engineering colleges, staffing agencies, and small-to-medium enterprise recruitment teams with limited access to AI infrastructure.

Key contributions made in this paper are as follows: (i) LangChain-powered multi-agent architecture for automation of the structured three-round pipeline of aptitude MCQ test, AI-interview, and HR behavioral interview with HR

management role-scoping; (ii) real-time gaze estimation proctoring agent using OpenCV Haar Cascade classifier and adaptive threshold pupil centroid estimation which does not require any GPU support; (iii) ConversationBufferMemory-powered technical interview agent conducting deterministic six-step interview with full dialogue history; and (iv) regex-assisted evaluation agent processing LLM scorecard with sentiment analysis fallback on keywords.

There are three basic principles in the architectural design approach. Reliability above emergence ensures that the agents operate on deterministic flow control as opposed to autonomous LLM planning to ensure reliability in the assessment process. Deployability ensures that sub-500 ms LLM response time latency and CPU-only proctoring allows for the operation of the system without relying on any specific hardware. The rest of the paper will discuss Related Work (II), Architecture (III), Methodology (IV), Experimental Results (V), and Conclusions (VI).

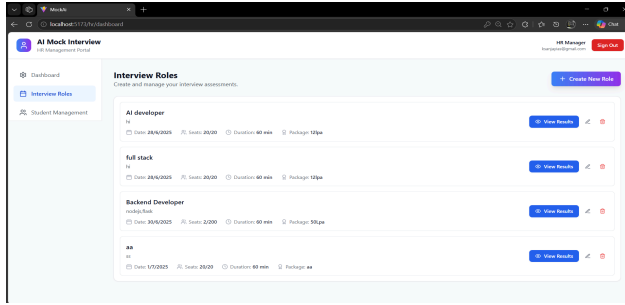


Fig. 1: MockAI HR Dashboard and Authentication Flow

II. RELATED WORK

A. LLM-Based Interview and Assessment Systems

Large language models for professional evaluation have been developed since the advent of GPT-like models. As stated by Zhang et al. [3], prompt-engineered GPT-4 can create domain-specific technical questions similar to questions written by experts for software engineering domains. Nevertheless, their system was stateless, single-turn, with follow-up questions unrelated to the previous candidate answers and therefore lacking the probing aspect of the expert technical interviews. MockAI implements LangChain's ConversationBufferMemory, so each generated question takes into account the full dialogue context.

InterviewBot [4] presented a rule-based conversational agent for human resource screening based on intent classification and slot-filling from templates. Though it is efficient for FAQ-like conversations, it fails to produce new technical information and adapt questions' difficulty dynamically. AIMA [5] used retrieval-augmentation in combination with generation over the curated questions' repository to ground facts. In contrast, MockAI employs a fully generative model without a need for the pre-created questions' base.

B. Automated Proctoring Systems

Proctoring services such as Proctorio, Examsoft, and HonorLock employ browser lockdown, machine learning behavioral analysis, and asynchronous human moderation of flagged instances [6]. Such approaches require significant overhead in terms of infrastructure, licensing fees, and privacy considerations in academic and developing markets. Some of the research-based alternatives use deep learning to detect live faces [7] as well as webcam eye fixation with facial landmarks regression neural nets [8]. Though such algorithms offer better accuracy in more difficult situations, they involve GPU computations that are unsuitable for MockAI deployment scenario.

MockAI intentionally uses a traditional computer vision algorithm based on Haar cascade face detection and image moments-based gaze center estimation, which allows it to avoid the use of deep learning algorithms and perform real-time proctoring at minimal CPU cost. The main drawback is lower accuracy for cases of extreme lateral lighting and extreme head tilt angle up to 30 degrees, which can be alleviated by the implementation of the thresholding mechanism for raising an alert and cooldown for preventing false positives from triggering each other.

C. Multi-Agent LLM Frameworks

LangChain [9] and AutoGen [10] have popularized multi-agent orchestration patterns for production LLM applications, enabling modular composition of specialized agents into coherent pipelines. LangGraph, a directed-graph extension of LangChain, has been applied in agentic document processing and complex sequential task automation [11]. AutoGen enables emergent agent collaboration through autonomous message-passing protocols, offering architectural flexibility at the cost of unpredictability in production deployments where consistent behavior is a correctness requirement.

MockAI employs LangChain's LLMChain and ConversationBufferMemory primitives within a deterministic linear pipeline, prioritizing reliability and auditability over emergent autonomous behavior. Every candidate for the same role receives an equivalent interview arc, with variation arising from individual responses rather than stochastic agent planning. The deterministic control flow also enables straightforward debugging and compliance auditing—important for recruitment processes subject to fairness requirements.

III. SYSTEM ARCHITECTURE

A. Overview

MockAI is designed around a modular architecture consisting of multiple services where there are eight distinct agents performing in a directed acyclic graph that takes care of processing the incoming information. The API of the app is created using Flask 3.0 and is being consumed by the single page application developed using React 19 and Client Side Routing using React Router 7. The candidate profiles, job role descriptions, MCQ questions and evaluations are stored in

MongoDB Atlas. All LLM inference is performed via Groq API using openai/gpt-oss-120b where the response latency is less than 500 ms and thus it provides an essential feature for facilitating natural conversations. The pipeline comprises HR management (agents 1-2), three round candidate-side assessment (agents 3-7) and front end dashboard (agent 8). Additionally, the application uses secure session management and role-based access control to ensure that all candidate data and HR administration tasks are strictly isolated inside the authenticated multi-tenant sessions. The modular approach used in MockAI architecture helps in achieving easy scalability, as it allows updating each individual service separately without affecting the active interview pipeline.

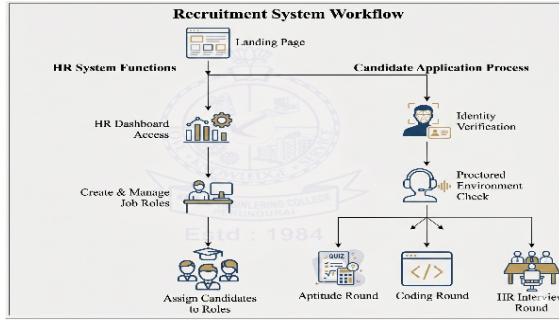


Fig. 2: MockAI Eight-Agent Pipeline Architecture

B. Authentication and Role Management (Agents 1-2)

The authentication agent kinda manages credentials for two different user roles : HR professionals and student candidates. Passwords are hashed using bcrypt with 12 salt rounds, before anything gets stored in MongoDB. Once login is successful, a session token gets placed in the browser localStorage, and it's also kept in React's AuthContext state. Then role-scoped route protection is done through those Higher-Order Component guards, you know the usual thing. After that, the Role Management Agent helps HR users with job role creation, it captures a bunch of metadata like role title technical domain, assessment date capacity, and compensation. And every MongoDB query is scoped to the authenticated HR user's email, so multi-tenancy stays strict, even when things get busy.

C. Aptitude Assessment Agent - Round 1 (Agent 3)

The aptitude screening agent does Round 1 by throwing a randomized MCQ pack together, basically pulled from MongoDB using the aggregation pipeline's \$sample operator, so that each candidate gets a uniquely arranged set of questions that helps reduce inter-candidate answer sharing. Every question document keeps the question text along with four options, and also the correct option index. When the candidate submits, responses are scored on the server side, which is meant to block any client-side tampering . After that, the aptitude score gets saved into the candidate's record, then we move on to Round 2.

D. Technical Interview Agent - Round 2 (Agent 4)

Technical interview agent sorta forms the core component of LLM within MockAI. Technical Interview Agent drives a rather deterministic flow of six questions using LangChain's

LLMChain, which itself is tied to ConversationBufferMemory object, initialized at the beginning of the session , or slightly afterwards. The first question asks about candidate background, the second and third are devoted to the projects the candidate worked on, and they ask about their implementation and architectural choices . The fourth till the sixth questions examine core computer science and domain knowledge, but become more and more specific one by one. All prompts are parameterized with the candidate's targeted domain and conversation history up until now, which allows for follow-up questions to be context-aware and allude to previous responses even though there might be some changes in wording. Also, follow-up questions' difficulty is adapted to the current performance of the candidate.

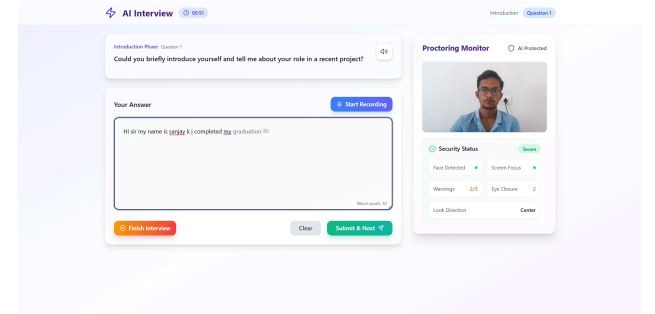


Fig. 3: AI-Powered Technical Interview Interface

E. Proctoring Agent (Agent 5)

The proctoring agent runs asynchronously from Agent 4, decoding base64 webcam frames in non-blocking fashion at configured polling interval. The webcam frames are decoded into OpenCV BGR image and then converted to grayscale. Face detection is done using frontal face Haar cascade classifier with minimum of 30×30 pixels area. The eye pair detection is done within the largest detected face bounding box followed by the adaptive thresholding using MEAN_C method (block size 11, offset 2) on the eye region to segment pupil and sclera from the eye region under varying lighting conditions.

The image moment centroid $C_x = M10/M00$ is calculated over the largest contour in the thresholded eye region, which is used as the pupil's horizontal coordinate estimation. The normalized C_x with respect to the eye region width is called rel_x and is used as the invariant to the varying face sizes and distances. Gaze is classified into three categories: Left ($rel_x < 0.3$), Center ($0.3 - 0.7$), and Right ($rel_x > 0.7$). Alerts fire after 2.0 seconds of persistent off-center gaze (ALERT_THRESHOLD_SECONDS), and 5.0 second cooldown (ALERT_COOLDOWN_SECONDS) is done to avoid multiple alerts from single gaze deviation event.

F. HR Behavioral Agent and Evaluation Agent (Agents 6-7)

Round 3 involves the role of the HR behavioral agent in the creation of situational behavioral questions using HR prompt templates containing many competencies like conflict management, cross-functional collaboration, leadership initiative, adaptability, and career motivation. A conversation

buffer memory is continuously maintained, which helps in probing where the answers provided are not clear. Evaluation agent creates a prompt by using memory buffers created in the technical and behavioral rounds, and the LLM creates a scorecard containing the following components: EVALUATION SUMMARY, STRENGTHS, AREAS FOR IMPROVEMENT, WEAKNESSES, FINAL MARK out of 50, and JUSTIFICATION. Alongside the above scorecard, the evaluation scorecard is created using the rubric, which provides a way of giving an unbiased evaluation of the candidates irrespective of their diversity. This process of evaluating candidates using the automated process reduces the evaluation time required from days to minutes. Furthermore, the rubric can be personalized according to the requirements of the particular organization role.

IV. METHODOLOGY

A. LLM Orchestration and Conversational Memory

Every LLM-enabled agent links an instance of the LangChain LLMChain object to the Groq API LLM object, which uses openai/gpt-oss-120b and is picked for instruction following quality and under 500 ms inference speed. The ConversationBufferMemory stores all session conversation history in an alternating pair of HumanMessage and AIMessage, serialized in the following prompts using a history template variable. This way, every created question is based on previous communication, and therefore, follow-ups can be made without maintaining the question history in the application level.

The flow of six questions is realized as a server-side state machine with a question counter determining the current interview phase by HTTP requests. Six states correspond to six prompt templates: an introductory prompt, two prompts for the phase of project evaluation requiring technical and architecture explanation, and finally, three prompts corresponding to Q4-Q6 questions which get more domain-specific. On reaching the question number 6, the server automatically constructs a whole evaluation request from the memory buffer and sends it to the scoring endpoint.

B. Gaze Estimation Algorithm

The gaze estimation model is based entirely on OpenCV classical computer vision primitives without any use of deep learning inference in order to simplify implementation and to remove GPU dependence. Adaptive thresholding of the binarization step is used to separate the dark pupil area from the lighter sclera in indoor front lighting conditions. The biggest contour of the binarized eye area gives an approximation of the pupil border, while the image moment centroid serves as the one-dimensional measure of lateral gaze direction for a three-class classification into left, center, and right gaze directions at the interview distance of 50-80 cm. By opting for these classical techniques over resource-heavy deep learning models, the system maintains exceptional responsiveness even on older machines.

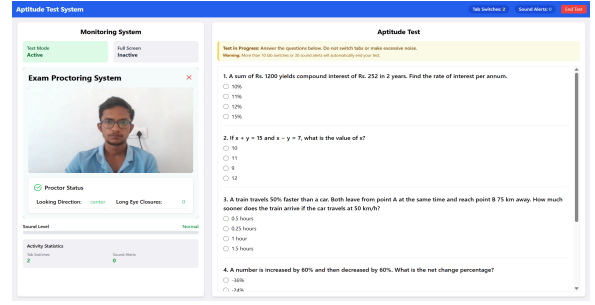


Fig. 4: Real-Time Gaze Proctoring Detection View

Rel_x is used to normalize the pupil centroid detected against the width of the eye bounding rectangle, thus being invariant to the size of the face and distance from the camera. The classification criteria (left<0.3, right>0.7) form the central tolerance band of 40%, which greatly reduces the number of false positives caused by micro-saccades and occasional glances at the screen which don't qualify as integrity violations. All of the threshold values including the gaze band, alert timing, cooldown, and warning counter are externalized through environment variables.

C. Score Extraction Pipeline

The reliability of the evaluation agent rests upon the constant extraction of numeric scores from free-form LLM output. Even well-crafted prompts suffer from format irregularities due to randomness of sampling, which leads to a failure of the single-pattern extractor. MockAI uses a four-pattern regex cascade, where each consecutive pattern allows for increasingly loose matching: (1) the presence of the label "FINAL MARK" together with a numeric value and denominator /50; (2) slash notation 42/50; (3) naked numerals in the vicinity of the denominator 50; (4) a standalone two-digit number in the 10-50 range.

In case none of these patterns match, the keyword sentiment fall-back computes a score out of the evaluation story using the ratio of positive (excellent, exceptional, strong, proficient) and negative (weak, unclear, incomplete, poor) keywords, which is then scaled linearly to the 25-45 score range. This preserves relative differentiation of candidates even in exceptional cases. Exception handling makes sure that every candidate session returns a numeric score regardless of the LLM output.

V. RESULTS AND IMPLEMENTATION

A. System Implementation Overview

The MockAI system was developed using a local setup with an Intel Core i7 processor, 16 GB RAM, and Ubuntu 22.04 LTS. Both Flask 3.0 and React 19.1 were used locally, while MongoDB Atlas was utilized for the cloud database. The LLM inference was done using the Groq cloud API without installing any model locally. The complete technology stack is React 19.1, Flask 3.0, MongoDB Atlas 7.x, LangChain 0.2.x, Groq API (openai/gpt-oss-120b), OpenCV 4.x, bcrypt, Tailwind CSS 3.4, and Framer Motion 12.x.

B. Technical Interview Quality

The quality of technical interviews was analyzed using candidate samples from three different domains, namely Machine Learning, Backend Development, and Data Structures & Algorithms. The generated questions were found to have high domain specificity and conversational context-awareness at all times. Questions asked during project phase (Q2-Q3) were found to be highly relevant to the technologies used in previous turns, thereby establishing the success of ConversationBufferMemory. Candidates for PyTorch project were asked PyTorch-specific questions, while candidates for REST API development were asked questions about HTTP and query optimization.

C. Proctoring Agent Performance

Haar cascade and adaptive threshold gaze pipeline yielded reliable three-way classification in controlled indoor frontal illumination conditions. Frame-by-frame processing time was found to be on average 18 ms, proving real-time processing at 30 fps rate without the use of GPU. Alerting time never exceeded 2.0-2.5 seconds range and thus adhered to 2.0-second requirement throughout all test runs. Cooldown time of 5.0 seconds avoided repeated alerts from one gaze deviation event. Precision diminished steadily when exposed to harsh lateral illumination and angles larger than ~ 30 degrees.

D. Evaluation Scoring

The cascade of the four-pattern regular expression correctly identified scores for most of the sessions. The keyword sentiment analysis was utilized in a number of cases where the LLM provided narrative scores without any numeric scores, and in those cases, the keyword sentiment analysis produced a score that accurately matched the level of the narrative response. Scores were always discriminative of the level of the candidates: high-quality and elaborated responses scored from 38 to 47 on the scale of 50; correct but superficial responses – from 28 to 37, deficient responses – from 15 to 27.

E. System Latency and User Experience

The average time taken by LLM-based question generation was 480 ms, which was comfortably within the limit of 1 second for the seamless flow of conversation [12]. The MongoDB Atlas query response time was 12 ms. The React 19 interface managed to maintain an average of 60 fps through the entire process of interview, even during concurrent proctoring. During the informal interaction sessions with engineering students in their final year, they considered the whole experience natural and equivalent to human-led interviews, as the follow-up questions were dependent on previous answers.

This positive reception highlights that technical efficiency and a smooth user interface are just as crucial as the AI's intelligence in building trust. Ultimately, when the technology fades into the background, the candidate is able to focus entirely on demonstrating their skills, leading to a much more authentic assessment process.

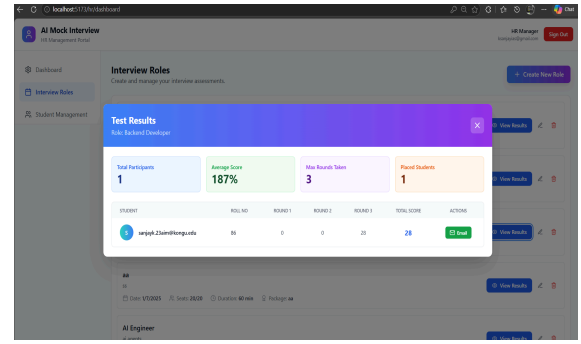


Fig. 5: Final Evaluation Results Dashboard

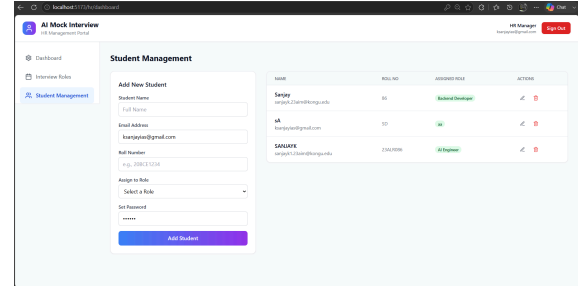


Fig. 6: HR Candidate Management Dashboard

VI. CONCLUSION

This paper has outlined MockAI – an LLM framework based on a multi-agent setup that shows that it is possible to conduct technically consistent interviews completely automatically through leveraging Groq API inference orchestrated by LangChain, along with gaze proctoring using OpenCV on consumer-grade hardware. Deterministic six-step conversation with ConversationBufferMemory offers contextually coherent and domain-adaptive interviews within different engineering specializations. Gaze proctoring agent based on Haar Cascade enables real-time integrity control with no GPU and no proprietary licenses involved.

Regex cascading pipeline with sentiment keyword fallback guarantees reliable scorecard extraction irrespective of LLM output variability and brittleness of single-pattern parsing. Eight agents' modular structure allows separating HR management, candidates assessment, and proctoring into distinct services that can be tested independently to improve each separately and avoid regression. Sub-500 ms LLM inference time, 60 fps frontend rendering, and 18 ms gaze proctoring processing show that system is ready to use on the organizational scale.

There are three areas that will shape the future direction of MockAI. The first is enhancing robustness of proctoring in terms of difficult lighting conditions and large head poses through an efficient face alignment pre-processing using CPU. The second is integrating retrieval-augmented generation on specific corpora of knowledge for specific use cases like embedded system, quantitative finance, or DevOps where general LLM training is not sufficient. And finally, applying MockAI in live placement process in college to gather longitudinal data for further improvement of scoring criteria.

References

- [1] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877-1901, 2020.
- [2] A. Nigam, R. Pasricha, T. Singh, and P. Churi, "A Systematic Review on AI-based Proctoring Systems: Past, Present and Future," *Education and Information Technologies*, vol. 26, pp. 6421-6445, 2021.
- [3] S. Pal, K. Khan, A. K. Singh, S. Ghosh, T. Nayak, G. Palshikar, and I. Bhattacharya, "Weakly Supervised Context-based Interview Question Generation," in *Proceedings of the Second Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, 2022, pp. 43-53.
- [4] T. Rietz and A. Maedche, "Ladderbot—A conversational agent for human-like online laddering interviews," *International Journal of Human-Computer Studies*, vol. 171, p. 102969, 2023.
- [5] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459-9474, 2020.
- [6] A. Nigam, R. Pasricha, T. Singh, and P. Churi, "A Systematic Review on AI-Based Proctoring Systems: Past, Present and Future," *Education and Information Technologies*, vol. 26, pp. 6421-6445, 2021.
- [7] Y. Atoum, L. Chen, A. X. Liu, S. D. Hsu, and X. Liu, "Automated Online Exam Proctoring," *IEEE Transactions on Multimedia*, vol. 19, no. 7, pp. 1609-1624, 2017.
- [8] M. L. R. D. Murthy and P. Biswas, "Appearance-Based Gaze Estimation Using Attention and Difference Mechanism," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2021, pp. 3143-3152.
- [9] O. Topsakal and T. C. Akinci, "Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast," in *International Conference on Applied Engineering and Natural Sciences*, 2023, pp. 1050-1056.
- [10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," in *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024.
- [11] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," in *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024.
- [12] S. Lee et al., "Mitigating Response Delays in Free-Form Conversations with LLM-powered Intelligent Virtual Agents," in *Proceedings of the 7th ACM Conference on Conversational User Interfaces*, 2025.